

2. L'objet du domaine

Étape 1 — EILMS : Domain Model & In-Memory Data Layer

Fichier concerné : `src/main/java/com/eilms/model/WarehouseItem.java`

C'est l'entité métier qui représente un article d'entrepôt. Elle impose les contraintes métier (encapsulation, validation) et un contrat d'identité clair (`equals/hashCode`).

```
public class WarehouseItem {
    private String id;
    private String name;
    private int quantity;

    public WarehouseItem() {}

    public WarehouseItem(String id, String name, int quantity) {
        if (quantity < 0) {
            throw new IllegalArgumentException("Stock quantity cannot be negative.");
        }
        this.id = id;
        this.name = name;
        this.quantity = quantity;
    }

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) {
        if (quantity < 0) {
            throw new IllegalArgumentException("Stock quantity cannot be negative.");
        }
        this.quantity = quantity;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        WarehouseItem that = (WarehouseItem) o;
        return Objects.equals(id, that.id);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

Explication détaillée

Attributs privés (`id`, `name`, `quantity`) : encapsulation classique — rien n'est accessible directement depuis l'extérieur, tout passe par des getters/setters. Cela protège l'objet d'une modification incontrôlée.

Constructeur avec validation : empêche de créer un objet dans un état invalide. Exemple : `new WarehouseItem("ITEM-1", "Câble", -3)` lève immédiatement une `IllegalArgumentException` au

lieu de créer un stock négatif incohérent.

setQuantity() avec la même règle : la validation n'est pas seulement à la création — elle s'applique aussi après coup. Exemple : `item.setQuantity(-5)` plantera plus tard dans le code, ce qui évite un bug silencieux qui ne serait découvert que bien après.

equals() basé uniquement sur id : deux `WarehouseItem` sont considérés « égaux » s'ils ont le même id, peu importe le nom ou la quantité. Exemple : `new WarehouseItem("ITEM-101", "A", 5).equals(new WarehouseItem("ITEM-101", "B", 99))` renvoie `true`.

hashCode() cohérent avec equals() : obligatoire en Java — deux objets égaux doivent avoir le même hashCode. Cela permet d'utiliser `WarehouseItem` en toute sécurité dans un `HashSet` ou comme clé d'un `HashMap`, sans doublons ni bugs.

Analyse pédagogique du professeur

Encapsulation & invariants : la validation de la quantité directement dans le constructeur et le setter empêche tout état de stock négatif illégal.

Contrat d'identité : implémenter `equals()` et `hashCode()` strictement sur le champ `id` permet un usage sûr dans les collections Java (`HashSet`, `HashMap`).